

In a distributed banking application, customers access their accounts and conduct transactions (deposits, withdrawals, and queries) across multiple branches of the bank. Each branch operates as an independent replica with a local balance, which must remain consistent with the balances at other branches. In this application, each process maintains a logical clock to ensure a consistent order of events based on the happens-before relationship as defined by Lamport's logical clock algorithm. The system should coordinate deposit and withdrawal requests from customers and propagate updates between branches while preserving causality across all processes.

The goal of this project is to model this distributed banking system using Python and gRPC to simulate interactions between customers and branch processes and to capture the events that occur on the customers side and the branch side to ensure a consistent view on the order of events. To do so, we must implement Lamport Logical Clocks to establish and validate the happens-before relationships in a distributed system. The main objectives are:

1. Simulate logical clocks in customer and branch processes.
2. Use Lamport's algorithm to synchronize events across processes, maintaining consistency and causality.
3. Track and output logical timestamps for all events, ensuring they respect the happens-before relation.

To set up the environment to implement this project, we had to use python's grpcio (version 1.67.0) and grpcio-tools (version 1.67.0) packages. The protobuf version accompanying the grpcio-tools was version 5.28.3. I additionally downloaded a protobuf 3 extension to my vscode setup for syntax validation. I then started implementing the project by defining the services needed to support the interactions between customer and branch, and branch and branch.

To support the protobuf messages, I defined an interface enum (it defined the types of transactions), and a result enum. The interface enum had fields WITHDRAW, DEPOSIT, and QUERY. The result enum had fields SUCCESS and FAILURE. I then defined messages CustomerMessage, BranchMessage, BranchEvent, AllBranchEvents, RequestEvents, and Reply. CustomerMessage are messages intended only to be sent by customers to a branch. It consisted of the fields id (for customer id), clock\_time(logical clock time at customer), request\_id(customer request id), interface (type of transaction a customer wished to perform), and amount (even though all fields is optional by default in a proto message, I wanted to clearly outline that amount is optional as it is only used in tandem with WITHDRAW and DEPOSIT interfaces). BranchMessage are messages intended only to be sent between branches. It consisted of the fields id(id of the requesting branch), request\_id(customer request id from requesting customer), interface (type of transaction to be performed at a branch), amount, and clock\_time(logical clock time at requesting branch). amount is not optional in this message because I designed the gRPC service such that banks only communicate with each other to propagate withdraw or deposit transactions. BranchEvent is a message that forms the basis for AllBranchEvents, i.e. the basis messages in a message that sends a list of messages of a certain

type. BranchEvent consists of the fields request\_id(customer request id of a branch event), clock\_time(logical clock time at branch of when the request was processed), interface(type of transaction customer performed), and comment(contains information on the origination/destination of an event). AllBranchEvents consist of repeated BranchEvent messages. RequestEvents is a message used by a customer to request all events that have occurred at a branch thus far and only consists of an id field of the requesting customer(however, it is not necessary). The Reply message is a message used to reply to both customers and branches. It consists of the fields interface, result (indicates whether a request was successful or not) and balance (optional, is only returned when the interface is query).

The three services I defined were Query, Propagate, and GetEventsAtBranch. Query is intended for customer to branch communication (customer sends a query request with CustomerMessage as argument and expects a Reply message). Propagate is intended for branch to branch communication (branch sends a propagate request with BranchMessage as argument and expects a Reply message). GetEventsAtBranch is intended for customer to branch communication where the purpose is to request all events at the customer's branch.

To be able to support customer Query RPCs, the Branch class defined in branch.py has \_createEvent, \_query, \_withdraw, and \_deposit methods as well as Query. When a branch object receives a Query rpc, it first sets its clock to the max of the value of its logical clock and the logical clock of the requesting entity and then increments it by 1. It then checks the interface of the CustomerMessage received. If it is QUERY, it creates a Reply message consisting of the interface initially received, the result of the request, and the balance of the branch. If the interface is either WITHDRAW or DEPOSIT, the branch propagates these requests to the other branches by first establishing communication channels to the list of branches the current branch is aware of (excluding itself) and sending a Propagate request to all those branches. Before each propagate request, the branch increments its logical clock value. During propagation, when the branch encounters its id in the list of branches, it performs the transaction locally without incrementing its logical clock value. This is because the branch logically must perform the transaction locally before attempting to propagate it. Additionally, before performing any type of transaction or propagating a transaction, the branch captures the event locally. Capturing an event consists of recording the request id of the transaction, the current logical clock value, the interface of the transaction, and a comment describing the nature of the event and then appending it to recvMsg, an instance variable that holds the list events that have occurred in an object (present in both Customer and Branch objects).

When a branch receives a Propagate RPC, it first updates its logical clock value by taking the max of its clock value and the clock value in the request and then incrementing it by 1. It then captures the event before it credits or debits (depending on the interface) its balance as per the amount in the BranchMessage. It finally responds with a Reply message. If at least one of the responses is a failure, \_withdraw and \_deposit would return false and we would construct a

Reply response with its result set to Failure. (nb. Since this is a simplified model, proper transaction semantics were not implemented. In the case of a proper model, all requests completed prior to the failure would have to be reversed.) If all communications with other branches in a branch's list of branches are successful, `_withdraw` and `_deposit` would return true and we would construct a Reply response consisting of the request interface and the result of the RPC call (successful). If the interface of a request isn't any of the three discussed, we respond with a failed Reply message.

The customer class defined in `customer.py` consists of the methods `_createEvent`, `createStub`, `executeEvents`, `requestEventsAtBranch`, and `returnMsgs`. `createStub` establishes a communication channel to a branch on a server in another process. All branches are on processes connecting to port `50,000+id` (this is to ensure that the ports used are very unlikely to be used by other processes when the servers are initially set up). Thus if a customer wants to communicate with the branch with its corresponding id, it can do so on port `50,000+self.id`. `executeEvents` first calls `createStub`. It then iterates over the list of events a customer ought to perform (this is stored in `events` as a list of dictionaries). For each event, we perform an RPC to a corresponding branch depending on the interface of the event. Before performing the RPC to the corresponding branch, we update the logical clock value of the customer by incrementing it by 1. After invoking the RPC, I capture the event and move on to the next.

To run the servers, we first read in the json file containing the input. From the, we retrieve each branch's set up information and a list of all branch ids. When we create a branch server, we first create a `Branch` object with the branch id, the starting balance, and the ids of the branches it can communicate with. We then instantiate a `grpc` server and add our branch service as a servicer. We then connect our server to a port from which it can listen to requests from before starting the server. I used python's multiprocessing module to instantiate each server on a different process and have it listen on ports according to their id (all branches are on processes connecting to port `50,000+id`).

To run the clients, we first read in the json file containing the input. From there, we retrieve each client's information and return a list of `Customer` objects using the information we retrieved. For this project, the clients must be run simultaneously so I made use of python's multiprocessing module. I also created a function that takes in a `Customer` object and invokes its `executeEvents` method. Since python operates on a pass by value scheme, I made use of multiprocessing's `Queue`. This was an additional parameter for the process function in order to pass back the `Customer` objects from the child processes to the main process.

The results stored in the output json file consist of the customer objects and their events of the transactions they completed, branch objects and their events of the transactions they completed, and finally event objects, detailing the type of object they were captured from, the id of the object, the logical clock value at the time of the event, the interface of the transaction, and

a comment describing the event in the format “event\_sent/event\_recv to/from <type of object> <id of object>”. This part was achieved by iterating over both customer and branch events and constructing the event information in the format given.